Defiende el servidor — Informe del taller

MLOps 2026-II · Grupo A · Nicol Tatiana Cerinza Tejada y Natalia Rios · Universidad Externado de Colombia

Código: github.com/Nicol-Tatiana-Cerinza-Tejada/Defiende-el-servidor · Modelos: huggingface.co/NicolTcT/defiende-el-servidor · Datos DVC: huggingface.co/datasets/NicolTcT/defiende-el-servidor-dvc

Construimos un clasificador binario normal/ataque con UNSW-NB15 y, sobre todo, la cadena de trazabilidad que permite recuperarlo: **commit Git** → **referencia DVC** → **run de MLflow** → **revisión de Hugging Face**. Todas las cifras de este informe se leen de los archivos de *evidencias/*.

1. Datos, auditoría y limpieza

Se usaron las particiones publicadas, descargadas del mirror fijado el 2026-10-24 (UTC). Los SHA-256 coinciden con el anexo del taller. Además de los nombres se comprobaron el número de filas, el esquema y las clases. No hay valores faltantes; el valor '-' de *service* se trata como una categoría más. Se excluyen como entradas *label* (objetivo), *attack_cat* (revela la respuesta) e *id*; quedan 42 variables (3 categóricas y 39 numéricas).

Archivo	Filas × cols	Normal / ataque	Repiten una entrada previa	Grupos contradictorios (filas)	SHA-256 (inicio)
training-set	175,341 × 45	56,000 / 119,341	74,301	229 (940)	bec7dd5ec88dc2a0…
testing-set	82,332 × 45	37,000 / 45,332	28,386	6 (14)	734fe6642edf758f…

Política de limpieza. Los duplicados exactos se conservan porque reflejan la frecuencia real del tráfico. Las 940 filas de los 229 grupos con entradas idénticas y etiquetas distintas se excluyen de entrenamiento y validación, porque en ellas las variables no determinan la etiqueta. No se elimina ninguna fila de test.

Particiones. Train y validación salen solo del archivo de entrenamiento. Se hace un *group split* por la huella de las variables predictoras: los grupos se barajan con la semilla 20261016 y se asignan a validación hasta llegar al 20 % de las filas. Así, ningún grupo de entradas idénticas queda repartido. Resultado: train 139,521 filas y validación 34,880. La asignación de cada fila está en *data/prepared/splits.csv*. Las particiones son las mismas en los cuatro experimentos, y codificadores, imputación y escalado se ajustan dentro del pipeline solo con train.

Cruces con test. 8,541 filas de test (10.37 %) tienen una entrada idéntica en el archivo de entrenamiento: 6,574 en train y 1,739 en validación. No se filtran, pero limitan las conclusiones.

Dos versiones de datos con DVC. *data/raw.dvc* guarda los dos CSV recibidos (md5 fe93051ae209…) y *data/prepared.dvc* guarda train/val/test/splits (md5 b3533724917e…). La transformación es *src/audit_prepare.py* y es determinista: en el clon nuevo regenera los mismos SHA-256. El remote es un dataset público de Hugging Face al que se accede por HTTP, así que *dvc pull* funciona sin credenciales y sin la caché original.

2. Cuatro experimentos, dos familias

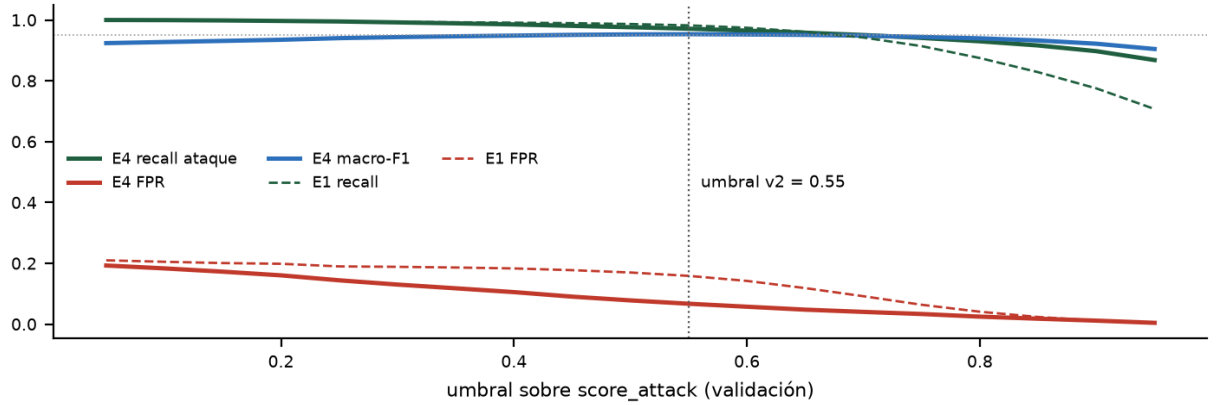
Cada run registra en MLflow la hipótesis, el cambio, los parámetros, la semilla, el commit Git, el md5 de DVC, las métricas de validación, la matriz de confusión, el barrido de umbrales, las puntuaciones de validación y el pipeline completo. La clase positiva es ataque (1). Cuando un denominador es cero, la métrica se reporta como 0. La tabla usa el umbral 0,5 en todos los casos.

Exp.	Familia / cambio	Hipótesis (resumen)	Recall	Prec.	FPR	Macro-F1	FP / FN	Run ID
E1	Logística base (one-hot + escala)	Referencia sencilla; colas pesadas limitarán el recall.	0.987	0.921	0.170	0.923	1963 / 312	c88acf26…
E2	Logística + log1p + balanced	log1p lineariza volúmenes; menos FPR que E1.	0.940	0.961	0.076	0.927	885 / 1409	8439d960…
E3	Árboles: RandomForest 200, prof. 20	Interacciones no lineales mejoran macro-F1.	0.978	0.957	0.089	0.950	1024 / 511	56cd27b0…
E4	Árboles: HistGB 300 it., 63 hojas	Boosting ≈ o > E3 con modelo más liviano.	0.976	0.962	0.078	0.952	902 / 557	83c62ca2…

Interpretación de errores. E1 detecta casi todos los ataques, pero alerta sobre el 17 % del tráfico normal (1 963 falsos positivos). E2 confirma en parte su hipótesis: reduce el FPR a la mitad, pero pierde 4,7 puntos de recall, así que el balanceo desplaza el error hacia los falsos negativos. Los árboles (E3 y E4) dominan a las logísticas en precision y FPR sin perder recall. E4 iguala o supera a E3 con un modelo de 2,2 MB, frente a 69 MB. Los run ID completos y la conclusión de cada experimento están en *experimentos.csv* y *mlflow/runs.json*.

3. Versiones v1 y v2 y elección del umbral

v1 es la primera solución funcional: E1 con el umbral por defecto 0,5. **v2** sale de una regla escrita en *configs/selection.yaml* antes de ver los resultados de E2–E4. Primero se elige la configuración con mayor macro-F1 de validación (E4). Luego, en su barrido de validación (pasos de 0,05), se toma el umbral con mayor macro-F1 entre los que mantienen un recall de ataque ≥ 0,95, porque para el equipo de seguridad perder ataques es lo más costoso. Resultado: umbral **0.55**. El test no intervino en ninguna decisión.



Validación (34 880 filas)	Recall ataque	Precision ataque	FPR	Macro-F1	Matriz [TN FP; FN TP]
v1 · E1 · umbral 0,50	0.987	0.921	0.170	0.923	[9606 1963; 312 22999]
v2 · E4 · umbral 0.55	0.971	0.967	0.067	0.953	[10794 775; 671 22640]

Qué mejora, qué empeora y qué aceptamos. v2 reduce las falsas alarmas de 1 963 a 775 (FPR de 17.0 % a 6.7 %) y sube la precision a 0.967. A cambio, deja pasar 671 ataques en lugar de 312: el recall baja 1,5 puntos. Aceptamos ese costo porque un FPR del 17 % haría inmanejable la cola de alertas, y el recall sigue por encima del mínimo de 0,95. Como el umbral se eligió en la misma validación, estas cifras de v2 son algo optimistas.

Publicación. Cada revisión de Hugging Face contiene *model.joblib* (pipeline completo de preprocesamiento y estimador), *config.json* (umbral, variables, SHA-256 del modelo, commit, md5 de DVC y run de MLflow), *requirements.txt* y la model card. Se identifican por SHA completo: v1 `2e405585467e01b576cbfd324488fa07174ef39e` y v2 `4ea2e9e7b2245d27a672e58b81d27574843615f7`, con tags v1/v2. En Git, los tags *v1-code* y *v2-code* marcan el código de cada solución.

4. Evaluación final en test

La selección se congeló en el commit `e378e2005e499e5f48ec153c421c7982e49573cd` (*seleccion.json*), anterior a cualquier uso de test. *evaluate_test.py* se niega a correr si esa decisión no está en un commit o si el test ya se evaluó. El modelo se descargó de Hugging Face por SHA y se verificó su hash. Se evaluó una vez sobre las 82,332 filas del test publicado, sin quitar ninguna. Las métricas se recalculan desde *predicciones-test.csv* (row_id, y_true, score_attack, y_pred), con la regla y_pred = 1 si score_attack ≥ 0,55.

v2 (E4, umbral 0,55)	Recall ataque	Precision ataque	FPR	Macro-F1	Accuracy	Matriz [TN FP; FN TP]
Validación	0.971	0.967	0.067	0.953	0.959	[10794 775; 671 22640]
Test (82 332 filas)	0.977	0.831	0.244	0.873	0.878	[27988 9012; 1063 44269]

El resultado empeora y lo conservamos. El recall se mantiene (0.977), pero el FPR sube de 6.7 % a 24.4 %: 9,012 conexiones normales alertadas. El análisis descriptivo posterior (*analisis-errores-test.json*, que no cambió el modelo) muestra dónde se concentran: las conexiones normales en estado INT se alertan en un 62.5 % y las FIN en un 25.6 %. Además, la proporción de INT y REQ en el tráfico normal es mayor en test que en entrenamiento. El recall más bajo es el de Fuzzers (84.3 %), la categoría de ataque que más se parece al tráfico normal. La validación salió del mismo archivo que el entrenamiento y no captó este cambio de distribución entre los dos archivos publicados. Es la lección principal del taller: una validación del mismo origen sobreestima el rendimiento.

5. Prueba de recuperación

scripts/prueba_recuperacion.sh se ejecutó en una carpeta nueva con un venv nuevo, un clon nuevo de GitHub (commit 5a29e8bfb8d8…), caché DVC vacía y un HF_HOME nuevo sin token. Pasos:

(1) instalar las dependencias fijadas; (2) *dvc pull* desde el dataset de Hugging Face, con las 6 huellas coincidentes; (3–4) descargar v1 y v2 por SHA completo y predecir sobre 30 filas fijas de validación (15 normales y 15 ataques); (5) comparar con las salidas guardadas antes de la recuperación. Resultado: **COINCIDEN para v1 y v2, con 0 etiquetas distintas y diferencia máxima de score 0,0** (tolerancia declarada 1e-09). Tiempo total: 3 min 24 s (incluye pip install). Además, en ese entorno, *train.py --exp E4_hist_gb* reprodujo las mismas métricas y un *model.joblib* con el mismo SHA-256 que v2. Entrada, salidas, hashes y log completo están en *evidencias/recuperacion/*.

6. Limitaciones

• UNSW-NB15 es tráfico sintético de laboratorio de 2015: no demuestra eficacia en una red actual.

• Hay cambio de distribución entre los archivos publicados de train y test: el FPR de test (24 %) es el número a considerar, no el de validación.

• El 10.37 % de las filas de test repite entradas del entrenamiento, lo que puede inflar algo las métricas de test.

• El umbral se ajustó en validación con una rejilla de 0,05; las puntuaciones no están calibradas como probabilidades.

• Las 940 filas contradictorias se excluyeron del entrenamiento; en test se evalúan y una parte es irreducible.

• MLflow se usó en local (sqlite). La evidencia consultable es la exportación en *evidencias/mlflow/*.

Referencias

Moustafa, N., & Slay, J. (2015). UNSW-NB15: a comprehensive data set for network intrusion detection systems. MilCIS 2015.

UNSW-NB15: https://research.unsw.edu.au/projects/unsw-nb15-dataset (atribución); mirror Nir-J/ML-Projects@44bbd02.

Documentación de Git, DVC, MLflow Tracking, Hugging Face Hub (descarga por revisión, model cards) y scikit-learn (common pitfalls).

Asistencia de IA: Claude Code (Claude Opus 5.5) para código, ejecución y borrador; el detalle está en entrega.md.